

CSE 423 — Compilers

Final Exam Scope and Study Guide

Instructor: Huixin Zhan

Exam Date: **May 15** Time: **1:30 PM – 2:30 PM** Room: **Cramer 107**

Logistics

- Duration: **60 minutes** (1:30 PM – 2:30 PM, May 15)
- Location: **Cramer 107**
- Allowed: **One (1) page cheat sheet, front and back**, handwritten or typed
- No calculators, laptops, phones, or other electronic devices
- Closed-book except for the cheat sheet above

1. Topics Covered

The final exam covers Lectures 11 through 20. Specifically:

1.1 Symbol Tables (Lecture 11)

- Purpose of a symbol table; what information is stored per entry (**sym**, **type**, scope, line number, parameters)
- Linked-list implementation: `struct symtab_entry` and `struct elem`
- `lookup()` and `insert()`: behavior, correctness, why `insert` must call `lookup` first
- Scope handling and nested scopes (chained or stacked symbol tables)

1.2 Type Construction from Syntax Trees (Lectures 12–13)

- The `btftp(nodeptr n)` (Build Type From Prototype) traversal
- Why a bottom-up post-order traversal is used to attach `n->type`
- Type cases: `INT`, `CHAR`, `IDENTIFIER (DONT_KNOW_YET)`, `*` (pointer), `PARAMDECL`, `PARAMDECLLIST` (tuple), `INITIALIZER_DECL` (function), `SIMPLE_DECLARATION`
- Where this fits in the GCC-style pipeline: `parse` → `symbol table` → `type construction` → `type checking` → `IR (GIMPLE)` → `optimization` → `codegen`

1.3 Synthesized vs. Inherited Attributes (Lecture 14)

- Distinction between synthesized (bottom-up) and inherited (top-down) attributes
- Worked example: `var a, b, c int` – propagating `int` to multiple identifiers
- Four-step process: compute type, copy type down, recurse, insert into symbol table
- When a variable's type is unknown at first parse and must be derived from context

1.4 Intermediate Code Generation / TAC (Lecture 16)

- Why intermediate code exists (portability, optimization, simplicity)
- Three-Address Code (TAC) form `x = y op z`
- Translating expressions and assignments into TAC; introducing temporaries $t_1, t_2, \dots$
- Arithmetic, memory (ADDR, LCON, SCON), control flow (GOTO, BLT/BLE/BGT/BGE/BIF/BIFN), procedure (PARM, CALL, RET)
- Memory regions and offsets: `glob:x, loc:0, heap:16`; pseudo-instructions `glob, proc, loc, lab, end`
- Semantic attributes `E.addr` and `E.icode`; helpers `gen`, `genlocal`, list concatenation

1.5 Basic Blocks and Control-Flow Graphs (Lecture 17)

- Definition of a basic block: single entry, single exit, no internal jumps
- Rules for boundary detection: first instruction, every label, every instruction after a jump
- Splitting TAC into basic blocks; drawing the corresponding flow graph
- “Label sandwich” for control structures (if/while)

1.6 Final Code Generation (Lecture 18)

- Five execution strategies (interpret source / parse tree / TAC, VM, native)
- Two big challenges: instruction selection and register allocation/assignment
- x86-64 register set; why register allocation is NP-complete; spilling
- Reading `gcc -S` output; calling conventions; `%rax` as return register
- LEAQ for arithmetic; why 64-bit code can be longer but faster

1.7 Code Optimization (Lecture 20)

- DAG (Directed Acyclic Graph) construction algorithm for a basic block
- Using a DAG to detect common subexpressions
- Topological sort to emit code; the array-aliasing pitfall (`a[i]` vs. `a[j] := y`)
- Optimization techniques:
 1. Constant folding
 2. Common subexpression elimination (explicit and implicit)
 3. Loop unrolling
 4. Algebraic identities and strength reduction (e.g., `y*16` $\rightarrow$ `y<<4`)
 5. Hoisting loop invariants
 6. Peephole optimization (register reuse, dead-store removal, fall-through, commutativity)
 7. Interprocedural optimization (IPO)

2. What You Are Expected to Do

By exam time, you should be able to:

1. Trace the contents of a symbol table after a sequence of declarations.
2. Annotate a syntax tree with `.type` fields, distinguishing synthesized vs. inherited.
3. Translate a small C-like fragment into Three-Address Code, introducing temporaries correctly.
4. Identify basic blocks in a TAC listing and draw the corresponding control-flow graph.
5. Build a DAG for a basic block and list every common subexpression it exposes.
6. Apply at least four optimization techniques by name to a given code fragment, and justify when each is *unsafe*.
7. Reason about register allocation tradeoffs and why the problem is NP-complete.
8. Read a short snippet of x86-64 assembly produced by `gcc -S` and explain what each instruction does.

3. Format of the Final

- **8 questions** weighted unequally; total 100 points.
- Mix of: short-answer, code-tracing, TAC translation, basic-block partitioning, DAG construction, optimization recognition, and one longer “put-it-all-together” problem.
- Partial credit is given. Show your work for any problem worth more than 5 points.

Study Tips

- Your one-page cheat sheet should hold: the TAC opcode table, the basic-block boundary rules, the DAG-build algorithm in 3 steps, and a checklist of optimization names.